

CSE5095: Types and Programming Languages

Homework 2: Type Safety

September 18, 2026

1 Booleans

Task 1. Write the inference rules for the dynamic semantics of Booleans and the if-else construct. You should have 2 new rules for the judgment $e \text{ val}$ and 3 new rules for the judgment $e \mapsto e'$.

Hint. The 2 value rules are the easy part: what are the only two Boolean literals, and by analogy with ValNum/ValString, what should their val rules look like (no premises)? For the 3 step rules on if e_1 then e_2 else e_3 : think about what has to happen *first* — the condition e_1 needs to be evaluated down to a value before you know which branch to take, so one rule should let you step e_1 while leaving the rest of the if-expression alone (compare to StepSearchAddLeft). The other 2 rules fire only once e_1 is already exactly true or exactly false — what should the whole if-expression step to in each case? (The assignment’s hint says exactly one of your three rules is a search rule — the other two should have no premises.)

Your answer:

Task 2. Evaluate if $\bar{1} + \bar{2} = \bar{4}$ then “Fake news” else “True fact” fully, showing each step.

Hint. Work from the inside out. First the addition inside the condition needs to reduce (StepAdd) — but it’s nested inside an equality test, which is itself nested inside your if-condition, so you’ll need the appropriate search rules to “reach in” at each level (StepSearchEqLeft, then your if-search rule from Task 1). Once the condition becomes a closed numeral comparison, figure out whether StepEq or StepNotEq applies. Once the condition is fully a Boolean literal, one of your two “actual” if-step rules from Task 1 fires and picks a branch — that branch, if it’s already a string literal, is a value and you’re done.

Your answer:

We also have the extended typing rules for equality testing and Booleans (given in the assignment):

$$\frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 = e_2 : \text{bool}} \text{ (TYPEEQ)} \quad \frac{}{\text{true} : \text{bool}} \text{ (TYPETRUE)} \quad \frac{}{\text{false} : \text{bool}} \text{ (TYPEFALSE)}$$

$$\frac{e_1 : \text{bool} \quad e_2 : \tau \quad e_3 : \tau}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau} \text{ (TYPEIF)}$$

Task 3. Give a derivation using the extended typing rules for `if  $\bar{1} + \bar{2} = \bar{4}$  then “Fake news” else “True fact”` : string.

Hint. Build the tree bottom-up, the same way the class notes built the derivation for `$\bar{0} + ((\bar{1} + \bar{2}) + \bar{3})$  : int`. Type the two numerals under the addition (TypeNum), combine with TypeAdd, then bring in `$\bar{4}$` and combine with TypeEq to get the condition typed `bool`. Separately, type each string branch with TypeString. Finally combine all three pieces with TypeIf — remember both branches must be assigned the *same* τ (here, `string`) for TypeIf to apply. Abbreviate rule names (e.g. TN, TA, TEq, TS, TIf) so the tree fits on the page; `\small` or `\scriptsize` around the whole derivation can help too.

Your answer:

Task 4. Prove the cases of the Preservation theorem for the new rules you added in Task 1.

Hint. Recall **Theorem (Preservation)**. If $e : \tau$ and $e \mapsto e'$, then $e' : \tau$. You’re doing induction on the derivation of $e \mapsto e'$, with one case per rule from Task 1. For the two non-search rules (where the if-expression steps directly to a branch), use *inversion* on TypeIf to read off what you already know about that branch’s type — no induction hypothesis needed there. For the search-rule case, you’ll need to apply your induction hypothesis to the sub-step of the condition, then re-apply TypeIf to reassemble the result. This should closely mirror the class’s proof of the StepSearchAddLeft case for the original Preservation theorem.

Your answer:

Task 5. State (you do not have to prove it) the new case of the Canonical Forms lemma for Booleans.

Hint. Look at the existing Canonical Forms statement for `int` and `string` (“if e `val` and $e : \tau$, then e looks like ...”). What are the *only* two values that could possibly have type `bool`, given the value rules from Task 1?

Your answer:

Task 6. Prove the cases of the Progress theorem for the new rules *TypeTrue*, *TypeFalse* and *TypeIf*. You may (and should) use the new case of Canonical Forms from Task 5.

Hint. Recall **Theorem (Progress)**. If $e : \tau$, then e `val` or there exists e' with $e \mapsto e'$. Induction is on the derivation of $e : \tau$. *TypeTrue*/*TypeFalse* should be one-line cases (compare to the *TypeNum*/*TypeString* cases in the notes). For *TypeIf*, apply the induction hypothesis to the sub-derivation of the *condition*’s typing judgment to get two branches (condition is a value, or condition steps); in the “condition is a value” branch, use your Task 5 Canonical Forms lemma to pin down exactly which of the two Boolean literals it must be, and then pick the corresponding rule from Task 1.

Your answer:

2 Division by zero

Task 7. Fill in the blanks in the typing rules for *EZ*.

Hint. For each operation, ask: *can* the result be zero, given what you know (or don’t know) about the operands? For $+$ and $-$: if both operands are only known to be “maybe zero,” can the sum/difference still come out to zero? (Try small examples, and remember integers can be negative.) For $*$: the two given rules already show the multiplication case is split depending on whether both factors are known-nonzero or not — think about *why* two nonzero integers multiplied together can never be zero. For $/$: what does the operational rule *StepDiv* require of the second operand in order to not get stuck? That tells you the required type of e_2 in *TypeDiv*. Then ask: even when e_1 and e_2 are both guaranteed nonzero, can integer division still produce a zero result? (Think about truncating division, e.g. small numerator, larger denominator.) That tells you the result type.

$$\begin{array}{c}
\frac{}{\bar{n} : \text{mz}} \text{ (TypeNum)} \qquad \frac{n \neq 0}{\bar{n} : \text{nz}} \text{ (TypeNumNZ)} \\
\\
\frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 + e_2 : ?} \text{ (TypeAdd)} \qquad \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 - e_2 : ?} \text{ (TypeSub)} \\
\\
\frac{e_1 : \text{nz} \quad e_2 : \text{nz}}{e_1 * e_2 : \text{nz}} \text{ (TypeMulNZ)} \qquad \frac{e_1 : \text{mz} \quad e_2 : \text{mz}}{e_1 * e_2 : \text{mz}} \text{ (TypeMulMZ)} \\
\\
\frac{e_1 : \text{mz} \quad e_2 : ?}{e_1 / e_2 : ?} \text{ (TypeDiv)}
\end{array}$$

Your answer:

Task 8. *Prove the Progress and Preservation theorems for EZ (cases TypeNum, TypeNumNZ, TypeDiv for Progress; StepAdd, StepSub, StepDiv for Preservation).*

Hint. These should closely mirror the class’s proofs of Progress and Preservation for the E language (TypeAdd/StepAdd cases), just swapping in your Task 7 rules and the EZ Canonical Forms lemma (given in the assignment) in place of the int/string versions. For the Progress/TypeDiv case in particular: apply your induction hypothesis to *both* e_1 and e_2 (nested, the way the class’s TypeAdd case does), use Canonical Forms to turn “value” into “some numeral,” and remember that StepDiv has a side condition ($n_2 \neq 0$) that your typing hypothesis on e_2 should give you for free. For Preservation, each case is short: invert on the relevant typing rule to pin down τ , then use TypeNum to retype the result.

Your answer:

Task 9 (Bonus). *Prove the StepMul case for Preservation. **Hint (from the assignment):** if you do inversion on the typing rules, there will actually be two rules you need to consider in your inversion, and thus two cases.*

Hint. A product $e_1 * e_2$ could have been typed by *either* TypeMulNZ or TypeMulMZ — inversion on the typing derivation doesn’t hand you a unique rule the way it did for StepAdd, so you’ll genuinely need

two sub-cases. In the TypeMulNZ sub-case, invert once more (on TypeNumNZ) to learn both numerals are nonzero, then think about why a product of two nonzero integers can't be zero. In the TypeMulMZ sub-case, there's much less to check.

Your answer:

Task 10. Give a syntactically valid expression in EZ that does not perform a division by zero, but is still not well-typed by the above typing rules. Informally suggest a change, addition, refinement, etc., to the type system that could make your expression well-typed (but would still rule out expressions that divide by zero).

Hint. Look for a gap between what's *true at runtime* and what the type system can *prove*. In particular, notice that TypeAdd and TypeSub (as you filled them in for Task 7) only ever conclude `mz` — there's no rule that lets a sum or difference be typed `nz`, no matter what its operands are. Now think of a division whose divisor is written as a sum or difference of two numerals that you, doing the arithmetic yourself, know is nonzero. For the refinement part: you don't need to design a whole new system, just gesture at what extra information the type system would need to track (e.g., sign, or a way to reason about specific known numeral values) to let that particular divisor be typed `nz`, while thinking about why your fix wouldn't accidentally let an actual division-by-zero back in.

Your answer:

3 One more wrap-up question

Task 11. How long (approximately) did you spend on this homework, in total minutes/hours of actual working time? Your honest feedback will help me with future homeworks.

Your answer: